String

String Class in C++ **2**
Size of String **6**
Common String Functions **6**
Concatenating Strings **7**
Comparing Strings **8**
Loop Through a String **10**
Using Index 10
Using Range-based for Loop 10
Modifying Characters **10**
Example Program **10**
Sample Output 11
string Class vs Character Array **11**
Advantages of the string Class **12**
Interview Questions **12**

String Class in C++

The **String class** in C++ is a **predefined class** provided by the C++ Standard Library to store and manipulate text (a sequence of characters).

It is defined in the **`<string>`** header file.

Instead of using a character array (`char[]`), C++ provides the `string` class, which automatically manages memory and offers many built-in functions for string operations.

Header File

#include <string>

or

#include <iostream>
#include <string>
using namespace std;

Why do we need the String Class?

In C, strings are stored using character arrays.

Example:

char name[20] = "Programming";

Problems with character arrays:

- Fixed size
- Manual memory management
- Difficult concatenation
- Difficult insertion/deletion
- Many library functions required

To solve these problems, C++ introduced the **`string`** **class**.

Declaring a String
Method 1

string name;

Method 2

string city = "Pune";

Method 3

string str("Hello");

Method 4

string s = "Embedded C++";

Example

```
#include <iostream>     // Provides cout for displaying output
#include <string>      // Provides the string data type

using namespace std;    // Allows us to use cout and string without std::

int main()
{
    // Create a string variable named 'name'
    // and initialize it with "Hello"
    string name = "Hello";

    // Display the value stored in the 'name' variable
    cout << name;

    // Return 0 to indicate successful program execution
    return 0;
}
```

Output
Pratidnya

Creating an Empty String

string str;

Initially,

str = ""

Reading a String

Using `cin`

string name;

cin >> name;

Input : John

Output : John

Limitation

`cin` stops reading when it encounters a space.

Input

John Smith

Stored string

John

Using `getline()`

string name;

getline(cin, name);

Input

John Smith

Stored string

John Smith

Displaying a String

cout << str;

Example

```
#include <iostream>
#include <string>
```

```
using namespace std;    // Allows us to use cout and string without std::

int main()
{
    // Create a string variable named 'str'
    // and initialize it with the text "Programming"
    string str = "Programming";

    // Display the string stored in the 'str' variable
    cout << str;

    // Return 0 to indicate successful program execution
    return 0;
}
```

Output : Programming

Accessing Characters

Using index

string str = "Programming";

cout << str[0];

Output : P

Example

```
for(int i = 0; i < str.length(); i++)
{
    cout << str[i] << " ";
}
```

Output : P r o g r a m m i n g

Size of String

str.length() or str.size()

Example

```
string str = "Programming";

cout << str.length();
```

Output

```
11
```

Both return the number of characters in the string.

Common String Functions

Function	Description	Example
`length()`	Returns number of characters	`str.length()`
`size()`	Same as `length()`	`str.size()`
`empty()`	Checks whether string is empty	`str.empty()`
`clear()`	Removes all characters	`str.clear()`
`append()`	Adds another string	`str.append(" C++")`
`push_back()`	Adds one character at the end	`str.push_back('!')`
`pop_back()`	Removes the last character	`str.pop_back()`
`insert()`	Inserts characters at a position	`str.insert(3, "ABC")`
`erase()`	Removes characters	`str.erase(2, 3)`
`replace()`	Replaces part of the string	`str.replace(0, 4, "Code")`
`find()`	Finds the position of a substring	`str.find("gram")`
`substr()`	Returns a substring	`str.substr(3, 5)`

`compare()`	Compares two strings	`str.compare(s2)`

Concatenating Strings

Using + operator

```
#include <iostream>
#include <string>

using namespace std;

int main()
{
    // Create the first string
    string s1 = "Hello";

    // Create the second string
    string s2 = "World";

    // Combine s1, a space, and s2
    // The + operator creates a new string
    string s3 = s1 + " " + s2;

    // Display the combined string
    cout << s3 << endl;

    return 0;
}
```

Output:

Hello World

Using append()

```
#include <iostream>
#include <string>
```

```
using namespace std;

int main()
{
    // Create a string
    string s1 = "Hello";

    // Append " World" to the existing string
    // append() modifies the original string
    s1.append(" World");

    // Display the modified string
    cout << s1 << endl;

    return 0;
}
```

Output: Hello World

Key difference

Method	Effect
`s3 = s1 + " " + s2;`	**Creates a new combined string**
`s1.append(" World");`	**Modifies the existing `s1` string**

Comparing Strings

```
#include <iostream>
#include <string>

using namespace std;

int main()
{
    // Create the first string
```

```
    string s1 = "Apple";

    // Create the second string
    string s2 = "Apple";

    // Compare both strings using the == operator
    // If both strings contain the same text, the condition is true
    if(s1 == s2)
    {
        // Display this message if both strings are equal
        cout << "Equal";
    }
    else
    {
        // Display this message if the strings are different
        cout << "Not Equal";
    }

    // Return 0 to indicate successful program execution
    return 0;
}
```

Output:

Equal

Important point

For C++ `string` objects:

s1 == s2

compares the **actual contents** of the two strings.

For example:

string s1 = "Apple";
string s2 = "Mango";

Output:

Not Equal

Loop Through a String

Using Index

```
for(int i = 0; i < str.length(); i++)
{
    cout << str[i];
}
```

Using Range-based `for` Loop

```
for(char ch : str)
{
    cout << ch << " ";
}
```

Output : P r o g r a m m i n g

Modifying Characters

```
string str = "hello";

str[0] = 'H';

cout << str;
```

Output : Hello

Example Program

```
#include <iostream>
#include <string>
using namespace std;

int main()
```

```
{
    string name;

    cout << "Enter your full name: ";
    getline(cin, name);

    cout << "Name : " << name << endl;
    cout << "Length : " << name.length() << endl;

    return 0;
}
```

Sample Output

Enter your full name: John Smith
Name : John Smith
Length : 10

`string` Class vs Character Array

Feature	`string` Class	Character Array (`char[]`)
Memory Management	Automatic	Manual
Size	Dynamic (can grow/shrink)	Fixed size
Easy Concatenation	Yes (`+`, `append()`)	Requires `strcat()`
Comparison	`==`, `compare()`	Requires `strcmp()`
Input with Spaces	`getline()`	`cin.getline()`
Safety	Safer, avoids many buffer overflow issues	More prone to overflow if not handled carefully
Built-in Functions	Many	Limited C library functions

Advantages of the `string` Class

- Easy to use.
- Dynamic size (grows and shrinks automatically).
- Supports direct assignment and comparison.
- Rich set of built-in member functions.
- Automatic memory management.
- Safer and more convenient than character arrays.
- Integrates well with STL containers and algorithms.

Practice Questions

1. What is the `string` class in C++?
2. Which header file is required to use `string`?
3. What is the difference between `length()` and `size()`?
4. Why is `getline()` preferred over `cin` for full-line input?
5. What is the difference between a `string` object and a `char` array?
6. How do you concatenate two strings?
7. How do you access and modify individual characters in a `string`?
8. What are some commonly used member functions of the `string` class?